Sistemas de Tiempo Real — Práctica 3

Ejercicio 4: Planificación Round Robin

Ingeniería en Computación

Conjunto de tareas y escala

Se cargan las cuatro tareas del Ejercicio 1 con igual prioridad. Con la escala del banco de medición (1 unidad = 4 ticks = 64 ms):

Tarea	Pin	C (unid.)	T (unid.)	C (ms)	T (ms)
A	PB0	2	12	128	768
B	PB1	2	24	128	1536
C	PB2	2	6	128	384
D	PB3	1	3	64	192

Factor de utilización $U = \frac{2}{12} + \frac{2}{24} + \frac{2}{6} + \frac{1}{3} = 0{,}9167$. Hiperperíodo $\mathrm{lcm}(12, 24, 6, 3) = 24$ unidades = 1536 ms, con 22 unidades de procesador ocupado y 2 de ocio.

Configuración del planificador

Según indica el enunciado, en `FreeRTOSConfig.h` de la biblioteca `configUSE_PREEMPTION` queda habilitado y `configUSE_TIME_ SLICING` se pone en 1, que es el valor que selecciona Round Robin entre tareas de igual prioridad. Son los dos únicos parámetros de la biblioteca que se tocan en toda la práctica; ningún otro archivo suyo se modifica. El firmware verifica la coherencia en tiempo de compilación y la imprime al arrancar, de modo que la terminal deja constancia de con qué configuración se tomó cada medición.

Limitación del simulador y cómo se resolvió

En el Ejercicio 3 se midió que, durante una ráfaga de espera activa de 64 ms, el contador de ticks no registra ninguno, cuando a 16 ms por tick deberían registrarse cuatro. Allí el desbalance se compensaba durante el ocio y el período salía exacto, porque con una sola tarea $U = 1/3$.

Con las cuatro tareas del Ejercicio 1 el ocio ya no alcanza, y el efecto se vuelve medible de forma directa. El reporte imprime `xTaskGetTickCount()` junto al reloj propio:

	Ticks por 3072 ms	Período efectivo	Contra el nominal
Nominal de la biblioteca	192	16,0 ms	—
Medido, Round Robin	94	32,7 ms	× 2,04
Medido, ráfagas atómicas	87	35,5 ms	× 2,22

El tick corre a menos de la mitad de la velocidad que la biblioteca supone, y el atraso depende de la carga. El error está del lado del tick y no del reloj propio: el analizador mide 16,05 ms sobre un tramo que el reloj propio cuenta como 16: si fuera el reloj propio el que corriera al doble, el analizador habría marcado 8.

La consecuencia es que `xTaskGetTickCount()` y todo lo que se apoye en él miden menos de la mitad del tiempo transcurrido. Se descartó modificar el port de la biblioteca, de modo que el banco se rediseñó para **no depender del tick**:

- **Reloj.** Un contador propio avanza 1 por cada milisegundo de CPU realmente consumido, dentro del mismo lazo de espera activa calibrado del Ejercicio 3. La tarea Idle consume el tiempo ocioso de la misma forma calibrada, de modo que la suma mide tiempo real.
- **Cuanto.** Lo produce la propia tarea: la ráfaga se parte en trozos de 16 ms y al final de cada uno se llama a `taskYIELD()`, que termina en `vTaskSwitchContext()` y rota la lista de listos igual que lo haría la interrupción del tick.
- **Activaciones.** Las libera un despachador con notificaciones de tarea, llamado al cerrar cada cuanto, de modo que una activación nunca se atrasa más de un cuanto. El reparto se hace con el planificador suspendido para que las cuatro queden liberadas en el mismo instante.

Instrumentación: los pines los escribe el núcleo. Una tarea decide cuándo empieza a usar el procesador, pero no cuándo lo pierde. Si cada tarea manejara su propio pin, una expropiación la dejaría con el pin en alto mientras otra ejecuta, y el analizador mostraría dos canales altos a la vez. Por eso los pines no los escribe la tarea sino el propio núcleo: FreeRTOS deja el gancho `traceTASK_SWITCHED_IN()` vacío detrás de una guarda `#ifndef`, de modo que definirlo desde el proyecto no modifica ningún archivo de la biblioteca. El núcleo lo invoca dentro de `vTaskSwitchContext()`, con las interrupciones deshabilitadas, y allí se escribe de una sola vez el nibble bajo de PORTB. Hay exactamente un canal en alto, el de la tarea que tiene el procesador, o ninguno si lo tiene la tarea

Idle: la superposición es imposible por construcción. El mismo gancho registra el cronograma en texto, de modo que los dos instrumentos muestran lo mismo.

Prueba de control: el reparto por tiempo del núcleo no rota. Queda por justificar por qué el cuanto lo fuerza la tarea si `configUSE_TIME_SLICING` está habilitado. La respuesta es experimental. Se compiló el mismo firmware, con la misma configuración de biblioteca, quitando únicamente la llamada a `taskYIELD()`:

	`configUSE_TIME_SLICING`		Ejercicio 5
	= 1, con `taskYIELD()`	= 1, sin `taskYIELD()`	= 0
Ráfagas	repartidas en cuantos	**atómicas**	atómicas
`fin1` de B	523 ms	**143 ms**	143 ms
`fin1` de C	529 ms	**271 ms**	271 ms
`fin1` de D	257 ms	**335 ms**	335 ms
`fin1` de A	507 ms	**705 ms**	705 ms
Plazos perdidos, C	1 por hiperp.	**0**	0
Plazos perdidos, D	1,13 por hiperp.	**2,99 por hiperp.**	3 por hiperp.

Sin `taskYIELD()`, y con el reparto por tiempo habilitado, el resultado es **idéntico al del Ejercicio 5**, que es la misma corrida con el reparto por tiempo deshabilitado. Las cuatro primeras ráfagas terminan en los mismos instantes, en el mismo orden, con los mismos plazos perdidos. Es decir que en este simulador `configUSE_TIME_SLICING` no tiene ningún efecto observable: la interrupción del tick no llega durante el cómputo y el núcleo nunca rota entre tareas de igual prioridad.

Un experimento controlado, cambiando una sola variable, es lo que permite afirmar que el `taskYIELD()` no es un agregado de conveniencia sino la única razón por la que hay un Round Robin que medir.

Consecuencia a declarar. Entre prioridades distintas la expropiación sigue siendo la del sistema operativo: la tarea Idle es expropiada por cualquier periódica que se libere. Entre tareas de igual prioridad, en cambio, el reparto lo fuerza la tarea y no el tick, aunque el mecanismo de cambio de contexto sea el mismo en los dos casos: `taskYIELD()` termina en `vTaskSwitchContext()`, que es exactamente la función que invoca la interrupción del tick.

a) El cuanto de tiempo

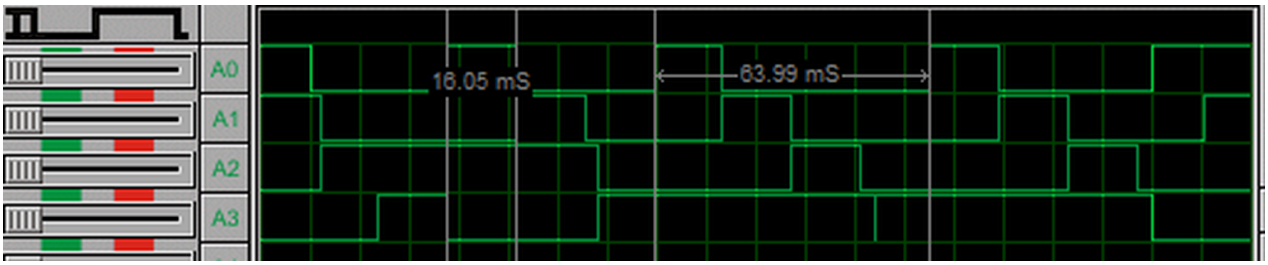


Figura 1: Canales A0–A3 (PB0–PB3 = tareas A–D). Cursores sobre un tramo de ejecución (**16,05 ms**) y sobre una vuelta completa de las cuatro tareas (**63,99 ms**).

El cuanto medido es de **16,05 ms**, contra 16,00 teóricos: error del 0,3 %. La vuelta completa mide **63,99 ms**, exactamente 4×16 ms, lo que confirma que cuando las cuatro tareas están listas el reparto es estrictamente equitativo y cíclico.

¿Coincide con el período del tick? Sí: 16 ms es el valor de `portTICK_PERIOD_MS` en esta biblioteca. Pero por lo explicado más arriba, en este banco el cuanto no lo *produce* la interrupción del tick, sino la tarea al ceder el procesador. El valor se eligió igual al del tick justamente para que el cronograma sea comparable con el que daría un reparto por expropiación.

b) Fin de la primera ráfaga

Con las cuatro tareas activadas simultáneamente en $t = 0$, el firmware registra el instante en que cada una completa su primera ráfaga (columna `fin1` del reporte):

Tarea	C (ms)	$n \cdot C$ teórico	`fin1` medido	Error
D	64	256	**257**	0,4 %
A	128	512	**507**	1,0 %
B	128	512	**523**	2,1 %
C	128	512	**529**	3,3 %

El modelo es inmediato: con $n = 4$ tareas repartiéndose el procesador en partes iguales, cada una avanza a un cuarto de velocidad y tarda $R \approx n \cdot C$ en completar su ráfaga.

El cronograma en texto permite además verificar el inciso (c) del Ejercicio 3, sumando los tramos de ejecución de cada primera ráfaga:

Tarea	Tramos de ejecución (ms)	Suma
A	$14 + 2 + 16 + 16 + 16 + 16 + 11 + 5 + 16 + 16$	**128**
B	$16 + 13 + 3 + 16 + 16 + 16 + 16 + 11 + 5 + 16$	**128**
C	$16 + 16 + 13 + 3 + 16 + 16 + 16 + 16 + 10 + 6$	**128**
D	$16 + 16 + 16 + 12 + 4$	**64**

Las cuatro suman exactamente su tiempo de cómputo pese a estar repartidas en hasta diez tramos: una tarea expropiada reanuda su ráfaga sin perder ni repetir el cómputo ya consumido, que es la condición que el enunciado exige al lazo de espera activa.

Los tramos que no miden 16 ms son los que el núcleo cortó por su cuenta en los bordes donde el despachador reanuda el planificador. Ocurren cada ≈ 63 ms y siempre en parejas que suman un cuanto: A hace 14 y luego 2, B hace 13 y luego 3, C hace 13 y luego 3. El cuanto sigue siendo de 16 ms; lo que cambia es que a veces se entrega partido en dos.

Conviene notar por qué las cuatro compiten durante todo el intervalo. D se reactiva a los 192 ms, cuando todavía no terminó su primera ráfaga (que recién completa a los 257), de modo que **nunca sale de la lista de listos**. El reparto es de a cuatro de punta a punta, y por eso A, B y C tardan $4 \times 128 = 512$ ms y no los 384 que habría dado un reparto de a tres.

c) Tabla de resultados

```
crear C -> 1
crear D -> 1
RAM libre=787
tarea activ perdidos Rmax jitter fin1
--- t=4480
A 6 0 508 1 508
B 3 0 524 1 524
C 12 3 529 1 529
D 24 3 257 11 257
--- t=7552
A 10 0 508 1 508
B 5 0 524 1 524
C 20 5 529 1 529
D 40 5 257 11 257
--- t=10624
```

Figura 2: Reporte periódico, un bloque cada 3072 ms (dos hiperperíodos). Columnas: tarea, activaciones, plazos perdidos, $R_{\text{máx}}$, jitter máximo y `fin1`, los cuatro tiempos en ms.

	Primer reporte ($t = 4480$ ms)		Corrida larga ($t = 480\,640$ ms)			
Tarea	Activ.	Perdidos	Activ.	Perdidos	% perdidos	T (ms)
A	6	0	626	0	0	768
B	3	0	313	0	0	1536
C	12	3	1252	313	**25,0 %**	384
D	24	5	2504	355	**14,2 %**	192

Las activaciones crecen entre reportes en +4, +2, +8 y +16, que es exactamente lo que corresponde a dos hiperperíodos: A se activa 2 veces por hiperperíodo, B 1, C 4 y D 8. Y los totales de la corrida larga coinciden al entero con t/T_i: $480640/768 = 625{,}8 \to 626$, y así las cuatro.

Tres observaciones sobre las tablas.

Sobre el instante crítico. En los primeros hiperperíodos $R_{\text{máx}}$ coincide con `fin1` en las cuatro tareas: el peor tiempo de respuesta se da en la primera ráfaga, con las cuatro liberadas juntas. Es el *instante crítico* del análisis clásico de planificabilidad, observado experimentalmente. Conviene precisar el alcance: marca el peor caso *del arranque*, no el máximo absoluto. Sobre 480 segundos los valores suben a 574, 575, 748 y 382 ms, porque las reactivaciones de C y D llegan a alinearse de forma más desfavorable que en el arranque.

Sobre el jitter. El despacho de activaciones ocurre al cerrar cada cuanto, de modo que una activación no puede atrasarse más de un cuanto. El máximo observado es de 15 ms contra un cuanto de 16: el valor medido queda justo por debajo del techo teórico, que es la forma en que una medición confirma un modelo. A y B quedan en

Tarea	$R_{\text{máx}}$ en el instante crítico	$R_{\text{máx}}$ en 480 s	Jitter máx.
A	507 ms	574 ms	1 ms
B	523 ms	575 ms	1 ms
C	529 ms	748 ms	15 ms
D	257 ms	382 ms	15 ms

1 ms porque sus instantes de activación son múltiplos de los de D y caen siempre en el mismo despacho que ella, en puntos donde el cuanto acababa de cerrarse.

Sobre la precisión del reloj propio. Al cabo de 480 segundos de reloj propio, el reloj de simulación de Proteus marca 523, un 8 % más. La diferencia es el tiempo que la tarea Idle pasa bloqueada transmitiendo por el puerto serie, que es tiempo real que el reloj no cuenta como CPU: 156 bloques de reporte a 9600 baudios son unos 17 segundos. No afecta ninguna de las medidas de este informe. Los `fin1` ocurren en los primeros 530 ms, donde el procesador está saturado y no se imprime nada, y el cuanto está medido con el analizador, que es independiente del reloj propio.

d) Por qué se pierden plazos con $U < 1$

Porque $U < 1$ garantiza únicamente que el trabajo *entra* en el procesador; no dice nada sobre *cuándo* entra. Round Robin reparte el procesador en partes iguales e ignora por completo los plazos.

Con n tareas listas, cada una recibe una porción $1/n$ del procesador. Una tarea cumple su plazo si le alcanza con esa porción, es decir si

$$n \cdot C_i \le T_i \qquad \Longleftrightarrow \qquad \frac{C_i}{T_i} \le \frac{1}{n}$$

La porción equitativa es $1/4 = 25$ %. Entonces:

Tarea	C_i/T_i	Porción recibida	$n \cdot C_i$	T_i	Resultado
A	16,7 %	25 %	512	768	cumple
B	8,3 %	25 %	512	1536	cumple
C	33,3 %	25 %	512	384	**pierde**
D	33,3 %	25 %	256	192	**pierde**

A y B piden menos que su porción equitativa y les sobra margen. C y D piden un tercio del procesador y reciben un cuarto: les falta trabajo por hacer cuando vence el plazo. La utilización total sigue siendo 0,9167 y el sistema no está sobrecargado; el problema es de reparto, no de capacidad.

e) Tarea más perjudicada

C y D son las dos perjudicadas, porque son las únicas que piden más que su porción equitativa. Entre ambas, los datos marcan a **C**.

Las dos piden lo mismo (1/3 del procesador) y las dos reciben lo mismo (1/4), pero el perjuicio no se reparte por igual. Sobre 480 segundos, C pierde 313 plazos de 1252 activaciones y D pierde 355 de 2504:

Tarea	Activaciones	Plazos perdidos	Proporción	Por hiperperíodo
C	1252	313	**25,0 %**	1,00
D	2504	355	**14,2 %**	1,13

C pierde uno de cada cuatro plazos, de forma exacta y estable: 313 plazos sobre 312,9 hiperperíodos es justo uno por hiperperíodo, y la proporción de 25 % se repite en todos los bloques del reporte desde el primero. D pierde algo más de uno por hiperperíodo, pero como se activa el doble de veces la proporción le queda casi la mitad.

La razón de fondo es que C necesita el doble de cómputo por activación (128 ms contra 64) con solo el doble de plazo, de modo que bajo reparto equitativo acumula su déficit en la mitad de oportunidades. D, al activarse más seguido, reparte una pérdida absoluta parecida entre el doble de trabajos.

El resultado general es el que motiva las políticas basadas en plazos: el reparto equitativo castiga justamente a las tareas más exigentes en tiempo, y trata igual a C, que tiene 384 ms para hacer 128 de cómputo, que a B, que tiene 1536 ms para hacer lo mismo.